

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

1. TITLE

Develop a Static Webpage Using Pure Semantic HTML Tags

2. AIM

To design and develop a static web application using pure HTML tags that structures an organized document hierarchy containing semantic layouts, tables, forms, and navigation links without external CSS.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the structural hierarchy of HTML5 semantic tags (<header>, <nav>, <main>, <article>, <aside>, <footer>, <section>).
 - To build a clean, accessible layout using pure HTML components without external styling.
 - To implement interactive web forms using <form>, <fieldset>, <legend>, <input>, <select>, and <textarea>.
 - To structure document layouts using borderless tables for multi-column presentation.
 - To process client-side form controls and navigation hyperlinks.
 - To display structured data using ordered () and unordered () lists.
-

4. THEORY

HTML (HyperText Markup Language) is the backbone of web development, defining the structural architecture of web documents. Pure semantic HTML utilizes tags that clearly communicate their purpose to browsers, screen readers, and search engines.

In this application, pure HTML is used to build a complete structured webpage:

- Semantic Containers (<header>, <nav>, <main>, <article>, <aside>, <footer>) define distinct content regions.
- Structural Tables (<table>, <tr>, <td>) align multi-column main and sidebar layouts.
- Form Controls (<input>, <select>, <textarea>) collect user details and inquiries.
- Navigation Links () provide smooth jump links within the document.

The basic flow of the application is:

Browser Request → HTML Parsing → Semantic Hierarchy Construction → Render Output

5. ALGORITHM / PROCEDURE

1. Create a new document named exp1.html.
 2. Initialize the document using standard HTML5 boilerplate syntax.
 3. Create a <header> section containing a structural table to position website logo and navigation links.
 4. Add internal navigation hyperlinks (, , etc.).
 5. Build a hero <section id="home"> with centered introductory headings.
 6. Construct a 2-column layout using a borderless <table> element.
 7. In the left column (<main>), add <article> blocks detailing core structural HTML tags.
 8. Create an interactive user inquiry form (<form id="contact">) inside an article.
 9. Add form inputs for Full Name (<input type="text">) and Email (<input type="email">).
 10. Add a dropdown menu (<select>) for inquiry topic selection.
 11. Include a multi-line message box (<textarea>) and action buttons (<input type="submit">, <input type="reset">).
 12. In the right column (<aside id="about">), add document reference lists using and .
 13. Add a <footer> section displaying copyright and semantic compliance declarations.
 14. Open exp1.html in a web browser to verify structural rendering.
 15. Verify form interaction and navigation link behavior across multiple test cases.
-

6. PROGRAM

exp1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>Pure Semantic HTML Static Webpage</title>
</head>
<body>
  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="0">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td>
          <td align="right">
            <nav>
              <a href="#home">Home</a> &nbsp;  |&nbsp;  
              <a href="#articles">Articles</a> &nbsp;  |&nbsp;  
              <a href="#about">About</a> &nbsp;  |&nbsp;  
              <a href="#contact">Contact Us</a>
            </nav>
          </td>
        </td>
      </tr>
    </table>
  </header>
  <hr>

  <!-- Hero / Showcase Section -->
  <section id="home">
    <center>
      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy that
browsers can cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>

  <hr>

  <!-- Main Content Layout using a Structural Table -->
  <table width="100%" border="0" cellspacing="0" cellpadding="20">
    <tr valign="top">

      <!-- Left Side: Main Content Column -->
      <td width="70%">
        <main id="articles">

          <article>
            <h2>1. Core Structural Tags</h2>
            <p>Published: July 7, 2026</p>

```

<p>When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <code><main></code>, <code><article></code>, and <code><section></code> tell screen readers and web scrapers exactly how information flows.</p>

<p>Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.</p>

</article>

<hr>

<article>

<h2>2. Native Interaction Elements</h2>

<p>Published: July 5, 2026</p>

<p>Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>

<h3>Interactive Demonstration Forms</h3>

<form id="contact" action="#" method="post">

<fieldset>

<legend>User Inquiry Form</legend>

<p>

<label for="name">Full Name: </label>

<input type="text" id="name" name="username" required
placeholder="John Doe">

</p>

<p>

<label for="email">Email Address: </label>

<input type="email" id="email" name="useremail" required>

</p>

<p>

<label for="topic">Inquiry Topic: </label>

<select id="topic" name="usertopic">

<option value="general">General Feedback</option>

<option value="technical">Technical Architecture</option>

<option value="academics">Academic Curriculum</option>

</select>

</p>

<p>

<label for="message">Message Details:</label>

<textarea id="message" name="usermessage" rows="5" cols="40"
placeholder="Type your notes here..."></textarea>

</p>

<p>

<input type="submit" value="Submit Form Data">

<input type="reset" value="Clear Entries">

</p>

</fieldset>

</form>

</article>

</main>

</td>

<!-- Right Side: Sidebar Column -->

```

        <td width="30%" bgcolor="#f4f4f4">
            <aside id="about">
                <h3>Document References</h3>
                <p>Static pages map data out directly without real-time database queries or
processor overhead.</p>

                <h4>Core Benefits:</h4>
                <ul>
                    <li>Instant browser painting</li>
                    <li>Low memory footprints</li>
                    <li>High data compatibility</li>
                </ul>

                <h4>Required Components:</h4>
                <ol>
                    <li>Document Type Definition</li>
                    <li>Semantic Wrapper Blocks</li>
                    <li>Data Input Control Interfaces</li>
                </ol>
            </aside>
        </td>

    </tr>
</table>

<hr>

<!-- Footer Section -->
<footer>
    <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
    </center>
</footer>

</body>
</html>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<!DOCTYPE html>	Declares the document as HTML5 for correct browser rendering.
<html>	Represents the root element of the HTML document.

<boby>	Contains all the visible content of the webpage.
<header>	Defines the semantic header area containing website title and navigation.
<nav>	Encloses top-level navigation anchor links.
<section>	Defines a separate section of the webpage.
<table>	Organizes and aligns webpage content into rows and columns
<main>	Specifies the primary content container of the document.
<article>	Encloses self-contained news items or independent text articles.
<form>	Captures user input data and specifies form submit actions.
<fieldset>	Groups related input fields visually with an explanatory legend.
<input>	Creates input fields such as text, email, submit, and reset controls.
<select>	Renders a dropdown list of inquiry topic choices.
<textarea>	Provides a multi-line text input field for detailed messages.
<aside>	Contains complementary sidebar notes, reference benefits, and lists.
<footer>	Defines the bottom page footer containing copyright information.
	Creates hyperlinks for navigation within the webpage.

8. TEST CASES AND EXECUTION DATA

The static HTML webpage was opened in the browser and verified across multiple test cases:

Test Case	Input Name	Test Input	Expected Result	Actual Result	Status
TC-01	Home / Articles Link	#home, #articles	Page scrolls to target section	Page scrolled correctly	Pass

TC-02	Form Required Field	Leave empty	Name	Browser demands Name input	Validation popup displayed	Pass
TC-03	Dropdown Selection	Select 'Technical'		Topic selected successfully	Topic selected correctly	Pass
TC-04	Clear Entries Button	Click Entries	Clear	All form fields reset	Fields cleared to empty	Pass
TC-05	Submit Button	Click Form	Submit	Form submits data to action	Form submission triggered	Pass

9. OUTPUT

Output 1: Pure Semantic HTML Static Webpage Execution

StaticWeb

[Home](#) | [Articles](#) | [About](#) | [Contact Us](#)

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below ↓](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprint

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Message Details:

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

10. RESULT

Thus, a static webpage was successfully designed using pure semantic HTML tags. Structural document hierarchy, form controls, navigation links, and lists were implemented and verified across multiple test cases.